

```
"""
```

```
Analyse du chômage au Maroc : import des données, calculs et graphiques.
```

```
Usage : python analyse_chomage_maroc.py
```

```
Dépendances : pandas, matplotlib (pip install pandas matplotlib requests)
```

```
Les données sont lues dans le dossier data/ (CSV construits à partir des notes du  
voir SOURCES.md). Une fonction optionnelle télécharge aussi la série OIT / Banque  
via l'API publique ; si Internet n'est pas disponible, le script utilise les vale  
"""
```

```
from pathlib import Path
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
BASE = Path(__file__).resolve().parent
```

```
DATA = BASE / "data"
```

```
FIG = BASE / "figures"
```

```
FIG.mkdir(exist_ok=True)
```

```
NAVY, RED, GREY = "#1F3A5F", "#C0392B", "#7F8C8D"
```

```
def charger_donnees():
```

```
    annuel = pd.read_csv(DATA / "chomage_annuel.csv")
```

```
    categories = pd.read_csv(DATA / "chomage_categories_2024_2025.csv")
```

```
    rupture = pd.read_csv(DATA / "rupture_methode_2025_2026.csv")
```

```
    return annuel, categories, rupture
```

```
def serie_banque_mondiale():
```

```
    """Optionnel : série OIT (modélisée) du chômage au Maroc via l'API de la Banq  
    try:
```

```
        import requests
```

```
        url = ("https://api.worldbank.org/v2/country/MA/indicator/SL.UEM.TOTL.ZS"  
              "?format=json&per_page=100")
```

```
        r = requests.get(url, timeout=15)
```

```
        r.raise_for_status()
```

```
        lignes = r.json()[1]
```

```
        df = pd.DataFrame([{"annee": int(x["date"]), "taux_oit": x["value"]}
```

```
                           for x in lignes if x["value"] is not None])
```

```
        return df.sort_values("annee")
```

```
    except Exception as e:
```

```
        print("API Banque mondiale indisponible, utilisation du CSV local :", e)
```

```
    return None
```

```

def fig_evolution(annuel):
    fig, ax = plt.subplots(figsize=(9, 5))
    ax.plot(annuel["annee"], annuel["taux_chomage_hcp"], marker="o", color=NAVY,
            linewidth=2.5, label="HCP (enquête nationale sur l'emploi)")
    oit = annuel.dropna(subset=["taux_chomage_oit_banque_mondiale"])
    ax.plot(oit["annee"], oit["taux_chomage_oit_banque_mondiale"], marker="s", li
            color=GREY, label="OIT / Banque mondiale (estimation modélisée)")
    for _, l in annuel.iterrows():
        ax.annotate(f"{l['taux_chomage_hcp']:.1f}", (l["annee"], l["taux_chomage_
            textcoords="offset points", xytext=(0, 8), ha="center", fonts
    ax.axvspan(2019.6, 2020.4, color=RED, alpha=0.12)
    ax.text(2020, 8.2, "Covid-19", ha="center", color=RED, fontsize=9)
    ax.set_title("Taux de chômage au Maroc, 2017-2025 (en %)", fontweight="bold")
    ax.set_ylabel("% de la population active")
    ax.set_ylim(7.5, 14.5)
    ax.grid(alpha=0.3)
    ax.legend(loc="lower right", fontsize=9)
    fig.text(0.01, 0.005, "Sources : HCP ; Banque mondiale/OIT. 2017-2018 à vérif
            fontsize=7, color=GREY)
    fig.tight_layout()
    fig.savefig(FIG / "fig1_evolution.png", dpi=170)
    plt.close(fig)

```

```

def fig_profils(categories):
    noms = ["Jeunes 15-24 ans", "Femmes", "Diplomes", "Milieu urbain",
            "Taux de chomage national", "Hommes", "Milieu rural", "Sans diplome"]
    d = categories[categories["indicateur"].isin(noms)].copy()
    d = d.sort_values("valeur_2025")
    fig, ax = plt.subplots(figsize=(9, 5))
    y = range(len(d))
    ax.barh([i + 0.2 for i in y], d["valeur_2024"], height=0.38, color=GREY, labe
    ax.barh([i - 0.2 for i in y], d["valeur_2025"], height=0.38, color=NAVY, labe
    ax.set_yticks(list(y))
    ax.set_yticklabels(d["indicateur"])
    for i, v in zip(y, d["valeur_2025"]):
        ax.text(v + 0.4, i - 0.2, f"{v:.1f}", va="center", fontsize=9, color=NAVY
    ax.set_title("Qui est le plus touché ? Taux de chômage par catégorie (en %)",
    ax.grid(axis="x", alpha=0.3)
    ax.legend()
    fig.text(0.01, 0.005, "Source : HCP, note sur la situation du marché du trava
            fontsize=7, color=GREY)
    fig.tight_layout()
    fig.savefig(FIG / "fig2_profils.png", dpi=170)
    plt.close(fig)

```

```

def fig_rupture(rupture):
    fig, ax = plt.subplots(figsize=(9, 5))
    x = range(len(rupture))
    ax.plot(x, rupture["taux_ancienne_methode_ENE"], marker="o", color=NAVY, line
            label="Ancienne méthode (ENE)")
    ax.plot(x, rupture["taux_strict_EMO2026"], marker="o", color=RED, linewidth=2
            label="Chômage strict (nouvelle enquête EMO2026)")
    for i, (a, b) in enumerate(zip(rupture["taux_ancienne_methode_ENE"],
                                   rupture["taux_strict_EMO2026"])):
        if pd.notna(a):
            ax.annotate(f"{a:.1f}", (i, a), textcoords="offset points", xytext=(0
                                     ha="center", fontsize=9, color=NAVY)
            ax.annotate(f"{b:.1f}", (i, b), textcoords="offset points", xytext=(0, -1
                                     ha="center", fontsize=9, color=RED)
    ax.set_xticks(list(x))
    ax.set_xticklabels(rupture["periode"])
    ax.set_title("Rupture de série : deux méthodes de mesure (en %)", fontweight=
    ax.set_ylim(8, 14.5)
    ax.grid(alpha=0.3)
    ax.legend(loc="upper right", fontsize=9)
    fig.text(0.01, 0.005, "Sources : HCP via Le Desk et FNH (2026).", fontsize=7,
    fig.tight_layout()
    fig.savefig(FIG / "fig3_rupture_methode.png", dpi=170)
    plt.close(fig)

```

```

def indicateurs(annuel):
    a = annuel.set_index("annee")["taux_chomage_hcp"]
    print("\n=== Indicateurs clés (HCP) ===")
    print(f"2019 -> 2024 : {a[2019]} % -> {a[2024]} % ({a[2024] - a[2019]:+.1f} p
    print(f"2019 -> 2020 (Covid) : {a[2020] - a[2019]:+.1f} point")
    print(f"2024 -> 2025 : {a[2025] - a[2024]:+.1f} point")
    print("Années de hausse :", [int(y) for y in a.index[1:] if a[y] > a[y - 1]])
    print("Années de baisse :", [int(y) for y in a.index[1:] if a[y] < a[y - 1]])

```

```

if __name__ == "__main__":
    annuel, categories, rupture = charger_donnees()
    bm = serie_banque_mondiale()
    if bm is not None:
        print(bm.tail())
    indicateurs(annuel)
    fig_evolution(annuel)
    fig_profils(categories)
    fig_rupture(rupture)

```

```
print("\nGraphiques enregistrés dans", FIG)
```